

Exercises Airflow

Exercise 1 – Create and Read a File

cat > exercise_1_dag.py

The screenshot shows a web-based code editor with a terminal on the left and a code editor on the right. The terminal displays the command `cat > exercise_1_dag.py` and the resulting file content. The code defines a DAG with two tasks: `create_file` and `read_file`. The `create_file` task writes a message to `/tmp/message.txt`, and the `read_file` task reads the message from the same file. The DAG is configured with a start date of `2026, 1, 1` and a schedule of `None`.

```
root@ubuntu:~# docker exec -it airflow bash
airflow@2be7cdd00ba:/opt/airflow$ cd /opt/airflow/dags
airflow@2be7cdd00ba:/opt/airflow/dags$ cat > exercise_1_dag.py
from datetime import datetime
from airflow import DAG
from airflow.operators.python import PythonOperator

def create_file():
    content = (
        "Welcome to Apache Airflow\n"
        "Learning DAGs\n"
        "Learning Task Dependencies\n"
    )
    with open('/tmp/message.txt', 'w') as f:
        f.write(content)

def read_file():
    with open('/tmp/message.txt', 'r') as f:
        print(f.read())

with DAG(
    dag_id='exercise_1_create_read_file',
    start_date=datetime(2026, 1, 1),
    schedule=None,
    catchup=False,
) as dag:

    task1 = PythonOperator(
        task_id='create_file',
        python_callable=create_file
    )

    task2 = PythonOperator(
        task_id='read_file',
        python_callable=read_file
    )

    task1 >> task2
```

The screenshot shows the same web-based code editor with the terminal displaying the execution of the DAG. The terminal output shows the DAG being loaded, the tasks being executed, and the DAG being completed. The `create_file` task is executed first, followed by the `read_file` task, which prints the message written by the first task.

```
python_callable=read_file
)
task1 >> task2
airflow@2be7cdd00ba:/opt/airflow/dags$ ls
exercise_1_dag.py
airflow@2be7cdd00ba:/opt/airflow/dags$ airflow dags reserialize
2026-06-23T10:16:33.173274Z [info] setup plugin alembic.autogenerate.schemas [alembic.runtime.plugins] loc=plugins.py:37
2026-06-23T10:16:33.173980Z [info] setup plugin alembic.autogenerate.tables [alembic.runtime.plugins] loc=plugins.py:37
2026-06-23T10:16:33.174728Z [info] setup plugin alembic.autogenerate.types [alembic.runtime.plugins] loc=plugins.py:37
2026-06-23T10:16:33.175241Z [info] setup plugin alembic.autogenerate.constraints [alembic.runtime.plugins] loc=plugins.py:37
2026-06-23T10:16:33.175777Z [info] setup plugin alembic.autogenerate.defaults [alembic.runtime.plugins] loc=plugins.py:37
2026-06-23T10:16:33.176158Z [info] setup plugin alembic.autogenerate.comments [alembic.runtime.plugins] loc=plugins.py:37
2026-06-23T10:16:33.176994Z [info] DAG bundles loaded: dags-folder [airflow.dag_processing.bundles.manager.DagBundlesManager] loc=manager.py:209
2026-06-23T10:16:33.792569Z [info] Filling up the DagBag from /opt/airflow/dags [airflow.dag_processing.dagbag.BundleDagBag] loc=dagbag.py:469
2026-06-23T10:16:33.905761Z [warning] The 'airflow.operators.python.PythonOperator' attribute is deprecated. Please use 'airflow.providers.standard.operators.python.PythonOperator'. [py.warnings] cat=py.warnings file=/opt/airflow/dags/exercise_1_dag.py lineno=4
2026-06-23T10:16:33.908272Z [info] Bulk-writing dags to db [airflow.serialization.definitions.dag] cont=1 loc=dag.py:197
2026-06-23T10:16:33.972728Z [info] Creating ORM DAG for exercise_1_create_read_file [airflow.dag_processing.collection] loc=collection.py:118
2026-06-23T10:16:33.978512Z [info] get next_dagrun_info_v2 [airflow.serialization.definitions.dag] last_automated_run_info=None loc=dag.py:1420 next_info=None
2026-06-23T10:16:33.979907Z [info] setting next_dagrun info [airflow.models.dag] loc=dag.py:792 next_dagrun=None next_dagrun_create_after=None next_dagrun_data_interval=None next_dagrun_data_interval_start=None next_dagrun_partition_date=None next_dagrun_partition_key=None
airflow@2be7cdd00ba:/opt/airflow/dags$ airflow dags list
2026-06-23T10:16:46.274834Z [info] setup plugin alembic.autogenerate.schemas [alembic.runtime.plugins] loc=plugins.py:37
2026-06-23T10:16:46.275866Z [info] setup plugin alembic.autogenerate.tables [alembic.runtime.plugins] loc=plugins.py:37
2026-06-23T10:16:46.276657Z [info] setup plugin alembic.autogenerate.types [alembic.runtime.plugins] loc=plugins.py:37
2026-06-23T10:16:46.277067Z [info] setup plugin alembic.autogenerate.constraints [alembic.runtime.plugins] loc=plugins.py:37
2026-06-23T10:16:46.277597Z [info] setup plugin alembic.autogenerate.defaults [alembic.runtime.plugins] loc=plugins.py:37
2026-06-23T10:16:46.278034Z [info] setup plugin alembic.autogenerate.comments [alembic.runtime.plugins] loc=plugins.py:37
dag_id | fileloc | owners | is_paused | bundle_name | bundle_version
-----|-----|-----|-----|-----|-----
exercise_1_create_read_file | /opt/airflow/dags/exercise_1_dag.py | airflow | True | dags-folder | None
airflow@2be7cdd00ba:/opt/airflow/dags$
```

In Airflow UI, execute the tasks by clicking trigger:

The screenshot shows the Apache Airflow web interface. The top navigation bar includes a search bar and a 'Trigger' button. The left sidebar contains navigation links: Home, Dags, Assets, Browse, Admin, Docs, and User. The main content area displays the details of a DAG run for 'manual_2026-06-23T10:18:32.622793+00:00'. The run is marked as 'Success'. Below the run details, there is a table of task instances:

Task ID	Map Index	State	Start Date	Try Number	Operator	Duration	Actions
read_file		Success	2026-06-23 15:48:38	1	PythonOperator	00:00:00.118	View Log, Copy, Refresh, Delete
create_file		Success	2026-06-23 15:48:33	1	PythonOperator	00:00:08.320	View Log, Copy, Refresh, Delete

cat /tmp/message.txt

The screenshot shows the KillerCoda playground interface. The top navigation bar includes a search bar and a 'Trigger' button. The left sidebar contains navigation links: Home, Dags, Assets, Browse, Admin, Docs, and User. The main content area displays the details of a DAG run for 'manual_2026-06-23T10:18:32.622793+00:00'. The run is marked as 'Success'. Below the run details, there is a table of task instances:

Task ID	Map Index	State	Start Date	Try Number	Operator	Duration	Actions
read_file		Success	2026-06-23 15:48:38	1	PythonOperator	00:00:00.118	View Log, Copy, Refresh, Delete
create_file		Success	2026-06-23 15:48:33	1	PythonOperator	00:00:08.320	View Log, Copy, Refresh, Delete

Or In Airflow UI, click on the read_file task which view logs to display the output.

The screenshot shows the Apache Airflow web interface. The top navigation bar includes links for Home, Dags, Assets, Browse, Admin, Docs, and a user profile. The main content area displays the 'read_file' task details for the DAG 'exercise_1_create_read_file'. The task status is 'Success'. Below the task details, there are tabs for Logs, Rendered Templates, XCom, Asset Events, Audit Log, Code, and Details. The 'Logs' tab is active, showing a log message source details section with a 'Pre Execute' section containing 8 log entries. The log entries are as follows:

Log ID	Timestamp	Level	Message
4	[2026-06-23 15:48:38]	INFO	Welcome to Apache Airflow
5	[2026-06-23 15:48:38]	INFO	Learning DAGS
6	[2026-06-23 15:48:38]	INFO	Learning Task Dependencies
7	[2026-06-23 15:48:38]	INFO	Done. Returned value was: None
8	[2026-06-23 15:48:38]	INFO	

Exercise 2 – Employee Salary Report

cat > exercise_2_dag.py

The screenshot shows the KILLERCODA playground editor. The left sidebar displays the 'Ubuntu 24.04' environment. The main editor area shows the code for 'exercise_2_dag.py'. The code is as follows:

```
airflow@2be7cdd00ba1:/opt/airflow/dags$ cat > exercise_2_dag.py
from datetime import datetime
from airflow import DAG
from airflow.operators.python import PythonOperator

def create_salary_file():
    content = "nihil, 45000\nPriya, 52000\nAmit, 61000\nSneha, 48000\n"
    with open('/tmp/employees.txt', 'w') as f:
        f.write(content)

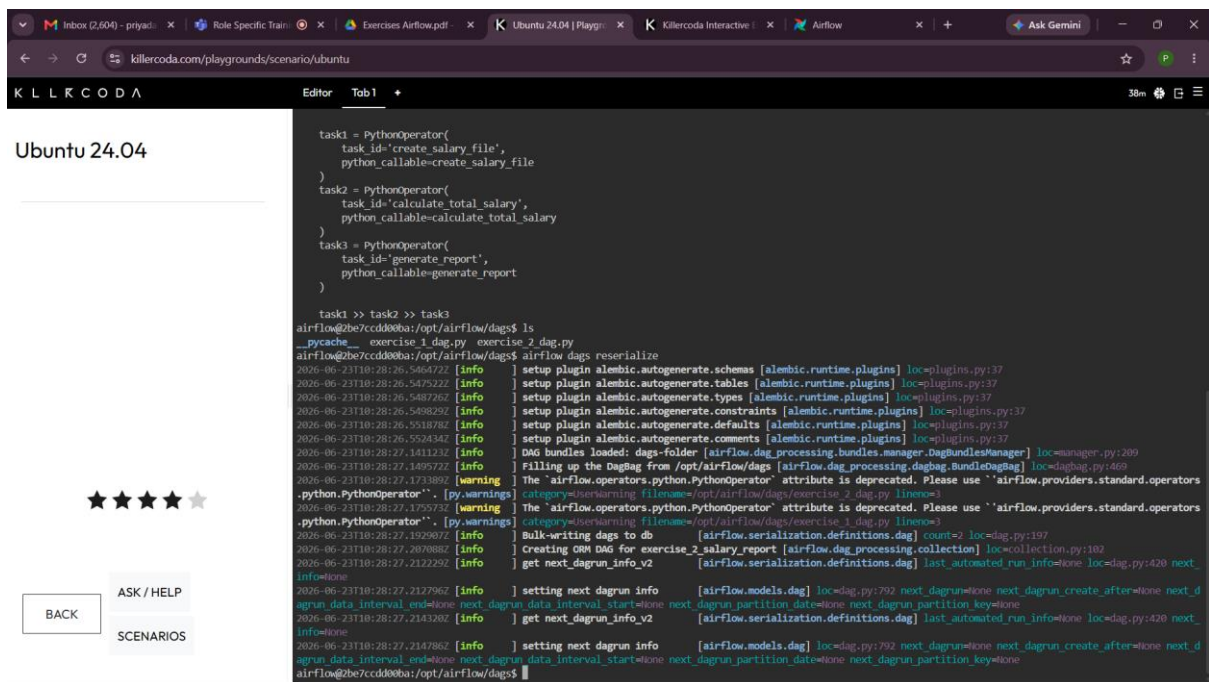
def calculate_total_salary():
    total = 0
    with open('/tmp/employees.txt', 'r') as f:
        for line in f:
            if line.strip():
                name, salary = line.split(',')
                total += int(salary.strip())
    print(f"Total Salary = {total}")
    return total

def generate_report(ti):
    total_salary = ti.xcom_pull(task_ids='calculate_total_salary')

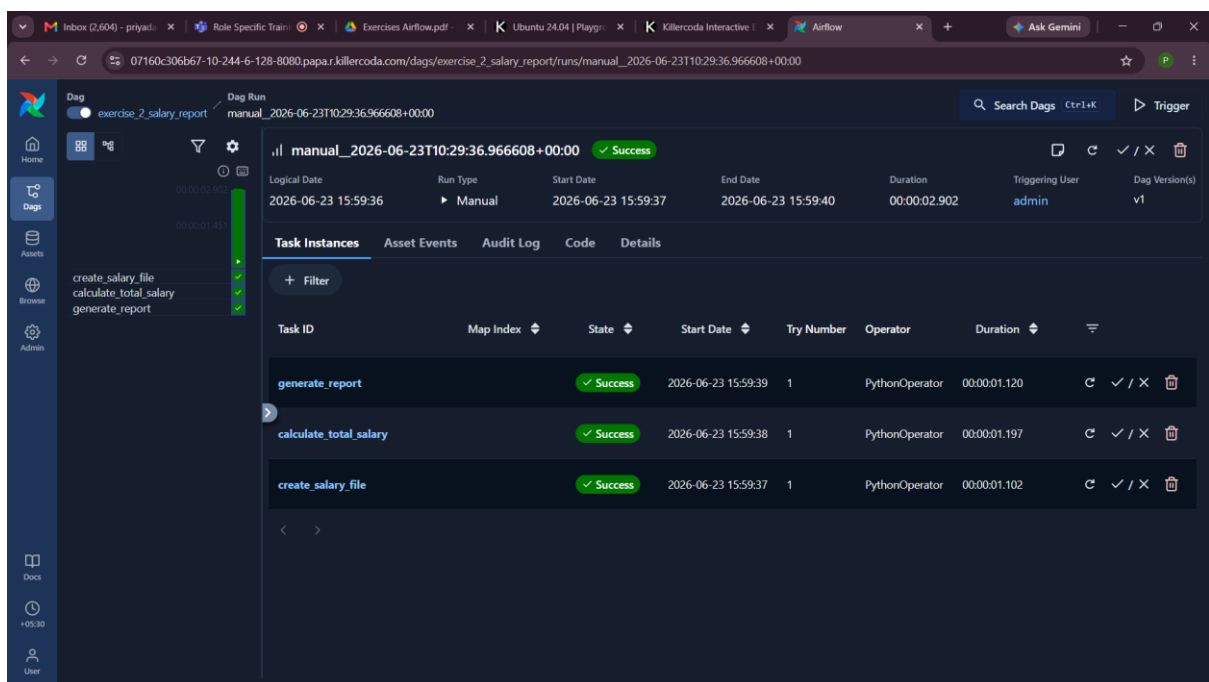
    with open('/tmp/employees.txt', 'r') as f:
        employees = [line for line in f if line.strip()]

    report_content = (
        "Salary Report\n"
        f"Employees = {len(employees)}\n"
        f"Total Salary = {total_salary}\n"
    )
    with open('/tmp/report.txt', 'w') as f:
        f.write(report_content)

with DAG(
    dag_id='exercise_2_salary_report',
    start_date=datetime(2026, 1, 1),
    schedule=None,
    catchup=False,
) as dag:
```



In Airflow UI, execute the tasks by clicking trigger:



Displaying output in Killercoda and Airflow UI:

```

airflow@2be7ccdd00ba:/opt/airflow$ cat /tmp/report.txt
Salary Report
Employees = 4
Total Salary = 206000
airflow@2be7ccdd00ba:/opt/airflow$
  
```

The screenshot shows the Airflow web interface. On the left, a sidebar contains navigation links: Home, Dags, Assets, Browse, Admin, Docs, and a clock icon. The main area displays the 'Dag Run' for 'exercise_2_salary_report'. The task 'calculate_total_salary' is highlighted with a green 'Success' status. Below this, a table shows the task's execution details:

Operator	Start Date	End Date	Duration	Dag Version
PythonOperator	2026-06-23 15:59:38	2026-06-23 15:59:39	00:00:01.197	v1

Below the table, the 'Logs' tab is active, showing log messages from the task's execution. The logs indicate that the task successfully calculated the total salary (206000) and generated a report.

Exercise 3 – Student Marks Processing

cat > exercise_3_dag.py

The screenshot shows the Airflow web interface with the 'exercise_3_dag.py' file open in the code editor. The code defines a DAG for processing student marks. The code is as follows:

```

airflow@2be7ccdd00ba:/opt/airflow/dags$ cat > exercise_3_dag.py
from datetime import datetime
from airflow import DAG
from airflow.operators.python import PythonOperator

def create_marks_file():
    content = "Math, 80\nScience, 75\nEnglish, 90\nPython, 95\n"
    with open('/tmp/marks.txt', 'w') as f:
        f.write(content)

def calculate_average():
    total_marks = 0
    count = 0
    with open('/tmp/marks.txt', 'r') as f:
        for line in f:
            if line.strip():
                subject, mark = line.split(',')
                total_marks += int(mark.strip())
                count += 1
    avg = total_marks / count
    print(f"Average = {avg}")
    return avg

def generate_result(ti):
    avg = ti.xcom_pull(task_ids='calculate_average')
    result = "PASS" if avg >= 40 else "FAIL"

    with open('/tmp/result.txt', 'w') as f:
        f.write(f"Average Marks = {int(avg)}\nResult = {result}\n")

with DAG(
    dag_id='exercise_3_marks_processing',
    start_date=datetime(2026, 1, 1),
    schedule=None,
    catchup=False,
) as dag:
    task1 = PythonOperator(
        task_id='create_marks_file',
        python_callable=create_marks_file
    )
    task2 = PythonOperator(

```

The interface also shows a sidebar with navigation links: BACK, ASK / HELP, and SCENARIOS. The top of the editor shows the file name 'exercise_3_dag.py' and the current directory path.

```

task_id='calculate_average',
python_callable=calculate_average
)
task3 = PythonOperator(
    task_id='generate_result',
    python_callable=generate_result
)

task1 >> task2 >> task3
airflow@2be7ccdd00ba:/opt/airflow/dags$ ls
__pycache__ exercise_1_dag.py exercise_2_dag.py exercise_3_dag.py
airflow@2be7ccdd00ba:/opt/airflow/dags$ airflow dags reserialize
2026-06-23T10:42:55.129116Z [info] ] setup plugin alembic.autogenerate.schemas [alembic.runtime.plugins] loc=plugins.py:37
2026-06-23T10:42:55.129308Z [info] ] setup plugin alembic.autogenerate.tables [alembic.runtime.plugins] loc=plugins.py:37
2026-06-23T10:42:55.129932Z [info] ] setup plugin alembic.autogenerate.types [alembic.runtime.plugins] loc=plugins.py:37
2026-06-23T10:42:55.130517Z [info] ] setup plugin alembic.autogenerate.constraints [alembic.runtime.plugins] loc=plugins.py:37
2026-06-23T10:42:55.131017Z [info] ] setup plugin alembic.autogenerate.defaults [alembic.runtime.plugins] loc=plugins.py:37
2026-06-23T10:42:55.131502Z [info] ] setup plugin alembic.autogenerate.comments [alembic.runtime.plugins] loc=plugins.py:37
2026-06-23T10:42:55.589870Z [info] ] DAG bundles loaded: dags-folder [airflow.dag_processing.bundles.manager.DagBundlesManager] loc=manager.py:209
2026-06-23T10:42:55.596656Z [info] ] Filling up the DagBag from /opt/airflow/dags [airflow.dag_processing.dagbag.BundleDagBag] loc=dagbag.py:469
2026-06-23T10:42:55.628536Z [warning] The 'airflow.operators.python.PythonOperator' attribute is deprecated. Please use 'airflow.providers.standard.operators
2026-06-23T10:42:55.631362Z [warning] category=UserWarning filename=/opt/airflow/dags/exercise_3_dag.py lineno=3
2026-06-23T10:42:55.631362Z [warning] The 'airflow.operators.python.PythonOperator' attribute is deprecated. Please use 'airflow.providers.standard.operators
2026-06-23T10:42:55.631362Z [warning] category=UserWarning filename=/opt/airflow/dags/exercise_1_dag.py lineno=3
2026-06-23T10:42:55.631362Z [warning] Bulk-writing dags to db [airflow.serialization.definitions.dag] count=3 loc=dag.py:197
2026-06-23T10:42:55.663930Z [info] ] Creating ORM DAG for exercise_3_marks_processing [airflow.dag_processing.collection] loc=collection.py:182
2026-06-23T10:42:55.668491Z [info] ] get next_dagrun_info_v2 [airflow.serialization.definitions.dag] last_automated_run_info=None loc=dag.py:420 next_
2026-06-23T10:42:55.669277Z [info] ] setting next_dagrun info [airflow.models.dag] loc=dag.py:792 next_dagrun=None next_dagrun_create_after=None next_d
2026-06-23T10:42:55.670845Z [info] ] get next_dagrun_info_v2 [airflow.serialization.definitions.dag] last_automated_run_info=None loc=dag.py:420 next_
2026-06-23T10:42:55.671032Z [info] ] setting next_dagrun info [airflow.models.dag] loc=dag.py:792 next_dagrun=None next_dagrun_create_after=None next_d
2026-06-23T10:42:55.672062Z [info] ] get next_dagrun_info_v2 [airflow.serialization.definitions.dag] last_automated_run_info=None loc=dag.py:420 next_
2026-06-23T10:42:55.672593Z [info] ] setting next_dagrun info [airflow.models.dag] loc=dag.py:792 next_dagrun=None next_dagrun_create_after=None next_d
2026-06-23T10:42:55.672593Z [info] ] get next_dagrun_info_v2 [airflow.serialization.definitions.dag] last_automated_run_info=None loc=dag.py:420 next_
airflow@2be7ccdd00ba:/opt/airflow/dags$

```

In Airflow UI, execute the tasks by clicking trigger:

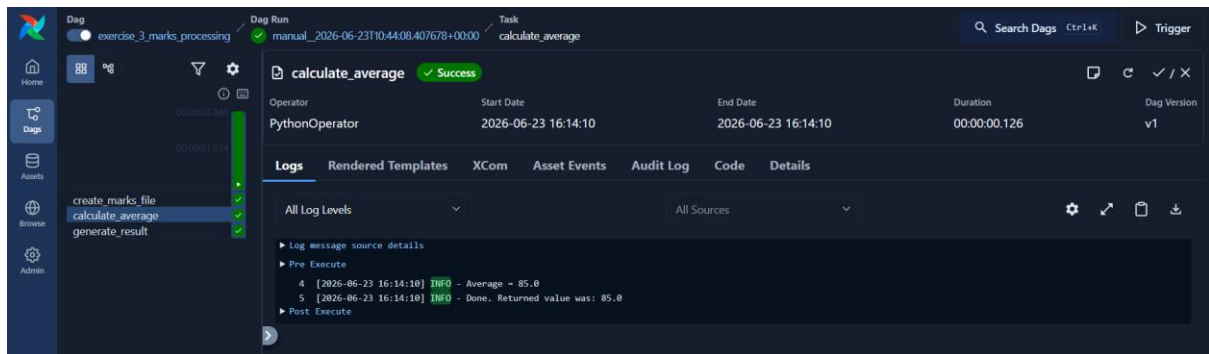
Task ID	Map Index	State	Start Date	Try Number	Operator	Duration
generate_result		Success	2026-06-23 16:14:11	1	PythonOperator	00:00:00.118
calculate_average		Success	2026-06-23 16:14:10	1	PythonOperator	00:00:00.126
create_marks_file		Success	2026-06-23 16:14:09	1	PythonOperator	00:00:00.109

Displaying output in Killercoda and Airflow UI:

```

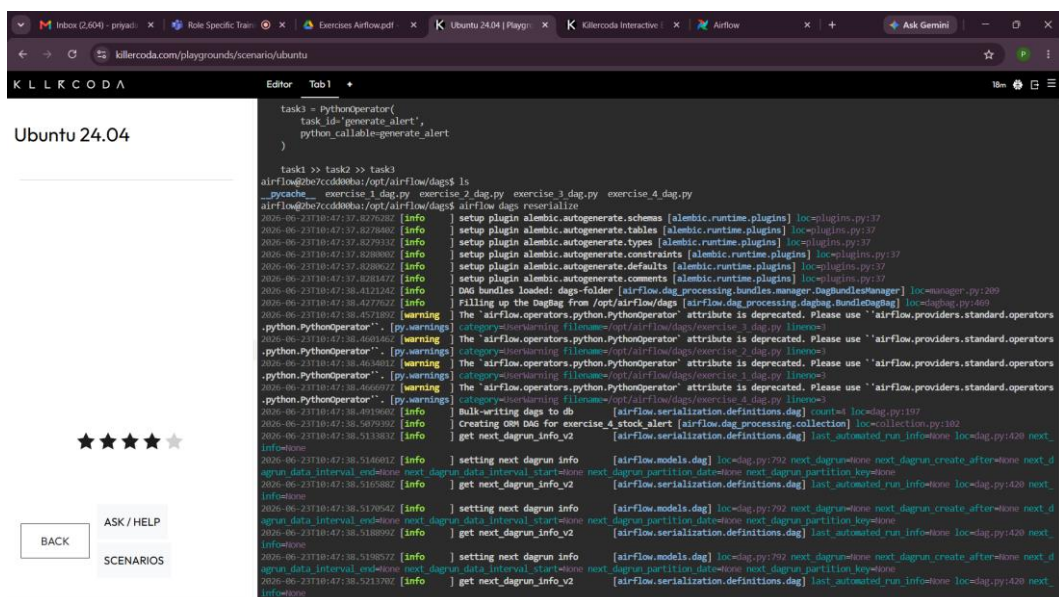
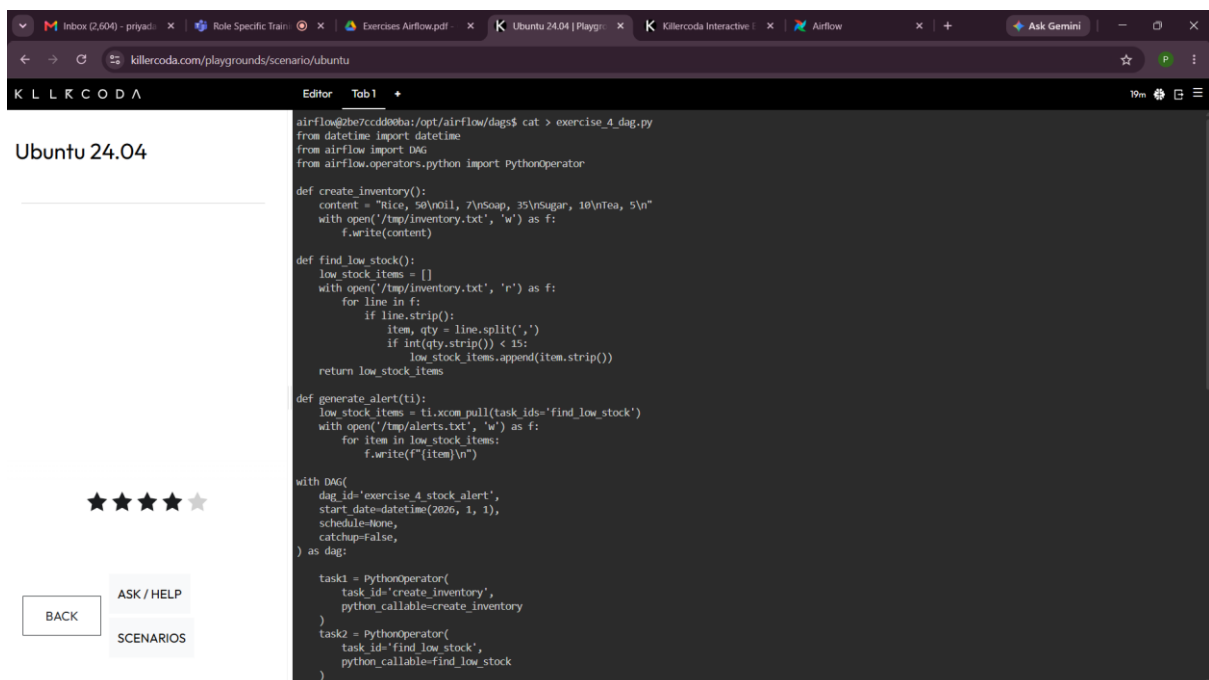
airflow@2be7ccdd00ba:/opt/airflow/dags$ cat /tmp/result.txt
Average Marks = 85
Result = PASS
airflow@2be7ccdd00ba:/opt/airflow/dags$

```

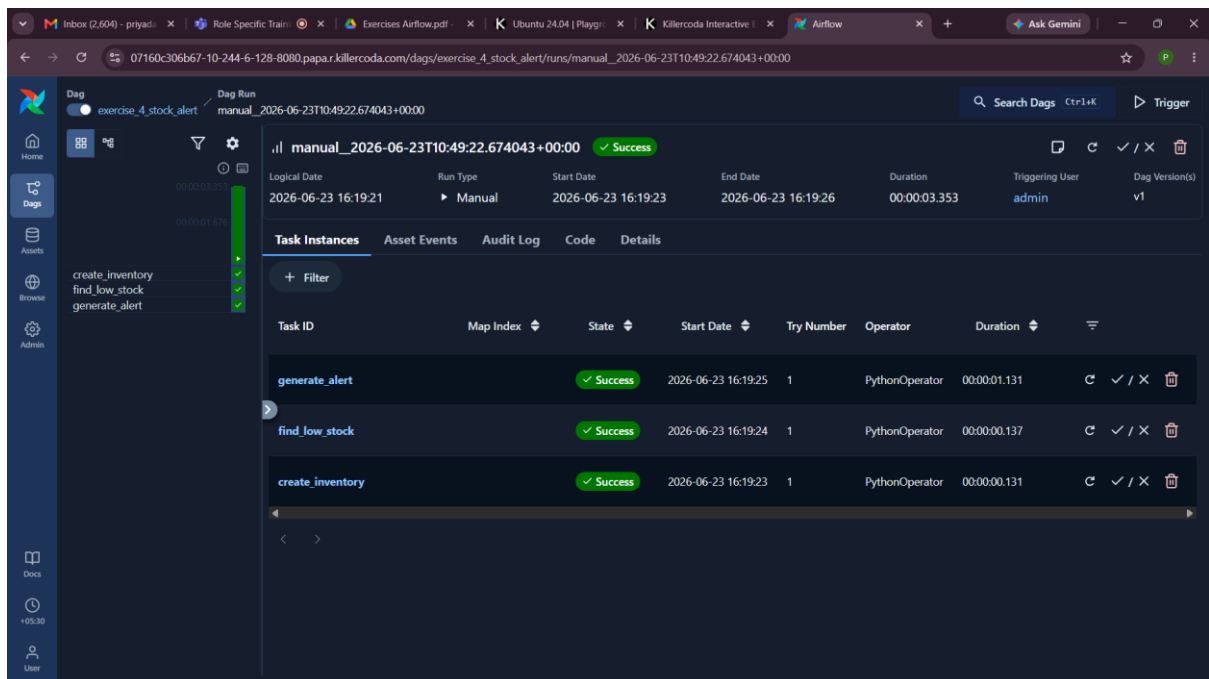



Exercise 4 – Product Stock Alert

cat > exercise_4_dag.py



In Airflow UI, execute the tasks by clicking trigger:

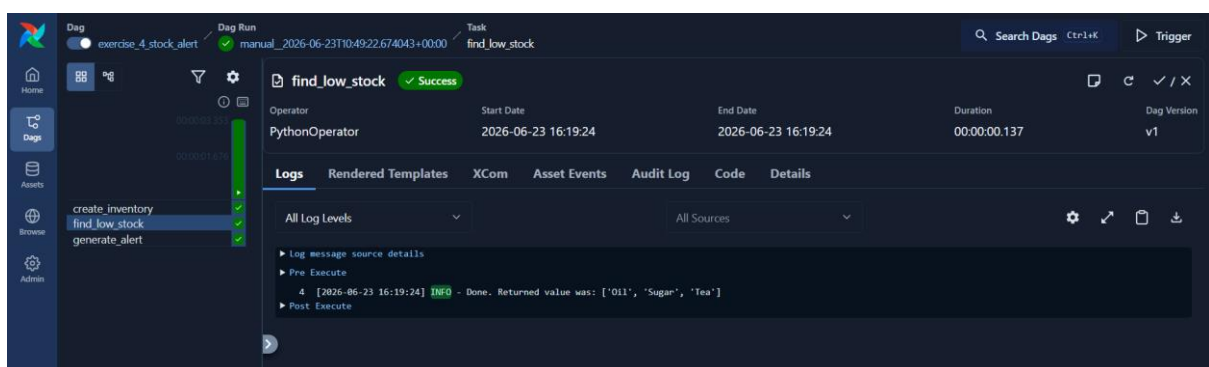


The screenshot shows the Airflow web interface for a DAG named 'exercise_4_stock_alert'. The 'Task Instances' tab is active, showing a table of task instances. The table has columns for Task ID, Map Index, State, Start Date, Try Number, Operator, and Duration. Three task instances are listed, all with a 'Success' state.

Task ID	Map Index	State	Start Date	Try Number	Operator	Duration
generate_alert		Success	2026-06-23 16:19:25	1	PythonOperator	00:00:01.131
find_low_stock		Success	2026-06-23 16:19:24	1	PythonOperator	00:00:00.137
create_inventory		Success	2026-06-23 16:19:23	1	PythonOperator	00:00:00.131

Displaying output in Killercode and Airflow UI:

```
Editor  Tab 1  +
airflow@2be7ccdd00ba:/opt/airflow/dags$ cat /tmp/alerts.txt
Oil
Sugar
Tea
airflow@2be7ccdd00ba:/opt/airflow/dags$
```



The screenshot shows the Airflow web interface for the 'find_low_stock' task. The 'Logs' tab is active, displaying the log output of the task. The log shows a message indicating that the task has completed successfully and returned the values ['Oil', 'Sugar', 'Tea'].

Exercise 5 – Attendance Report

cat > exercise_5_dag.py

Ubuntu 24.04

★★★★★

ASK / HELP

SCENARIOS

```
airflow@2be7cdd00ba:/opt/airflow/dags$ cat > exercise_5_dag.py
from datetime import datetime
from airflow import DAG
from airflow.operators.python import PythonOperator

def create_attendance():
    content = "Rahul, Present\nPriya, Present\nVinit, Absent\nSneha, Present\nKiran, Absent\n"
    with open('/tmp/attendance.txt', 'w') as f:
        f.write(content)

def count_present():
    count = 0
    with open('/tmp/attendance.txt', 'r') as f:
        for line in f:
            if "Present" in line:
                count += 1
    return count

def count_absent():
    count = 0
    with open('/tmp/attendance.txt', 'r') as f:
        for line in f:
            if "Absent" in line:
                count += 1
    return count

def generate_summary(ti):
    present = ti.xcom_pull(task_ids='count_present')
    absent = ti.xcom_pull(task_ids='count_absent')
    total = present + absent

    summary = (
        f"Total Students = {total}\n"
        f"Present = {present}\n"
        f"Absent = {absent}\n"
    )
    with open('/tmp/attendance_report.txt', 'w') as f:
        f.write(summary)

with DAG(
    dag_id='exercise_5_attendance_report',
```

Ubuntu 24.04

★★★★★

ASK / HELP

SCENARIOS

```
start_date=datetime(2026, 1, 1),
schedule=None,
catchup=False,
) as dag:
    task1 = PythonOperator(
        task_id='create_attendance',
        python_callable=create_attendance
    )
    task2 = PythonOperator(
        task_id='count_present',
        python_callable=count_present
    )
    task3 = PythonOperator(
        task_id='count_absent',
        python_callable=count_absent
    )
    task4 = PythonOperator(
        task_id='generate_summary',
        python_callable=generate_summary
    )

    task1 >> [task2, task3] >> task4
airflow@2be7cdd00ba:/opt/airflow/dags$ ls
__pycache__ exercise_1_dag.py exercise_2_dag.py exercise_3_dag.py exercise_4_dag.py exercise_5_dag.py
airflow@2be7cdd00ba:/opt/airflow/dags$ airflow dags reserialize
2026-06-23T10:55:16.167066Z [info] setup plugin alembic.autogenerate.schemas [alembic.runtime.plugins] loc=plugins.py:37
2026-06-23T10:55:16.167485Z [info] setup plugin alembic.autogenerate.tables [alembic.runtime.plugins] loc=plugins.py:37
2026-06-23T10:55:16.167742Z [info] setup plugin alembic.autogenerate.types [alembic.runtime.plugins] loc=plugins.py:37
2026-06-23T10:55:16.167939Z [info] setup plugin alembic.autogenerate.constraints [alembic.runtime.plugins] loc=plugins.py:37
2026-06-23T10:55:16.168065Z [info] setup plugin alembic.autogenerate.defaults [alembic.runtime.plugins] loc=plugins.py:37
2026-06-23T10:55:16.168265Z [info] setup plugin alembic.autogenerate.comments [alembic.runtime.plugins] loc=plugins.py:37
2026-06-23T10:55:16.773318Z [info] DAG bundles loaded: dags-folder [airflow.dag_processing.bundles.manager.DagBundlesManager] loc=manager.py:209
2026-06-23T10:55:16.779528Z [info] Filling up the DagBag from /opt/airflow/dags [airflow.dag_processing.dagbag.BundleDagBag] loc=dagbag.py:469
2026-06-23T10:55:16.819183Z [warning] The "airflow.operators.python.PythonOperator" attribute is deprecated. Please use "airflow.providers.standard.operators.python.PythonOperator". [py.warnings] category=UserWarning filename=/opt/airflow/dags/exercise_3_dag.py lineno=3
2026-06-23T10:55:16.822264Z [warning] The "airflow.operators.python.PythonOperator" attribute is deprecated. Please use "airflow.providers.standard.operators.python.PythonOperator". [py.warnings] category=UserWarning filename=/opt/airflow/dags/exercise_2_dag.py lineno=3
2026-06-23T10:55:16.825602Z [warning] The "airflow.operators.python.PythonOperator" attribute is deprecated. Please use "airflow.providers.standard.operators.python.PythonOperator". [py.warnings] category=UserWarning filename=/opt/airflow/dags/exercise_1_dag.py lineno=3
2026-06-23T10:55:16.829137Z [warning] The "airflow.operators.python.PythonOperator" attribute is deprecated. Please use "airflow.providers.standard.operators.python.PythonOperator". [py.warnings] category=UserWarning filename=/opt/airflow/dags/exercise_4_dag.py lineno=3
```

In Airflow UI, execute the tasks by clicking trigger:

The screenshot shows the Airflow web interface. On the left, there's a sidebar with navigation options: Home, Dags, Assets, Browse, and Admin. The main area displays a DAG run for 'exercise_5_attendance_report' with a status of 'Running'. Below this, there's a table of task instances. The tasks listed are 'generate_summary', 'count_absent', 'count_present', and 'create_attendance', all of which are marked as 'Success'.

Task ID	Map Index	State	Start Date	Try Number	Operator	Duration
generate_summary		Success	2026-06-23 16:26:15	1	PythonOperator	00:00:00.118
count_absent		Success	2026-06-23 16:26:14	1	PythonOperator	00:00:00.225
count_present		Success	2026-06-23 16:26:14	1	PythonOperator	00:00:00.240
create_attendance		Success	2026-06-23 16:26:13	1	PythonOperator	00:00:00.097

Displaying output in Killercode:

```

Editor  Tab 1  +

airflow@2be7ccdd00ba:/opt/airflow/dags$ cat /tmp/attendance_report.txt
Total Students = 5
Present = 3
Absent = 2
airflow@2be7ccdd00ba:/opt/airflow/dags$

```

Exercise 6 – CSV Processing

The screenshot shows the Killercode playground interface. On the left, there's a sidebar with navigation options: Home, Dags, Assets, Browse, and Admin. The main area displays a Python script for processing CSV files. The script defines functions for creating, reading, and calculating revenue from a CSV file.

```

airflow@2be7ccdd00ba:/opt/airflow/dags$ cat > exercise_6_dag.py
import csv
from datetime import datetime
from airflow import DAG
from airflow.operators.python import PythonOperator

def create_csv():
    content = (
        "product, quantity, price\n"
        "Laptop, 2, 70000\n"
        "Mouse, 5, 500\n"
        "Keyboard, 3, 1200\n"
    )
    with open('/tmp/sales.csv', 'w') as f:
        f.write(content)

def read_csv():
    rows = []
    with open('/tmp/sales.csv', mode='r') as f:
        reader = csv.DictReader(f)
        for row in reader:
            clean_row = {k.strip(): v.strip() for k, v in row.items()}
            rows.append(clean_row)
    return rows

def calculate_revenue(ti):
    rows = ti.xcom_pull(task_ids='read_csv')
    revenues = {}
    total_revenue = 0

    for row in rows:
        prod = row['product']
        qty = int(row['quantity'])
        price = int(row['price'])
        rev = qty * price
        revenues[prod] = rev
        total_revenue += rev

    return {"breakdown": revenues, "total": total_revenue}

def create_summary(ti):

```

```
data = ti.xcom_pull(task_id='calculate_revenue')
breakdown = data['breakdown']
total = data['total']

with open('/tmp/sales_summary.txt', 'w') as f:
    for prod, rev in breakdown.items():
        f.write(f'{prod} = {rev}\n')
    f.write(f'Total Revenue = {total}\n')

with DAG(
    dag_id='exercise_6_csv_processing',
    start_date=datetime(2026, 1, 1),
    schedule=None,
    catchup=False,
) as dag:
    task1 = PythonOperator(task_id='create_csv', python_callable=create_csv)
    task2 = PythonOperator(task_id='read_csv', python_callable=read_csv)
    task3 = PythonOperator(task_id='calculate_revenue', python_callable=calculate_revenue)
    task4 = PythonOperator(task_id='create_summary', python_callable=create_summary)

    task1 >> task2 >> task3 >> task4

airflow@e380ea5772f7:/opt/airflow/dags$ airflow dags reserialize
2026-06-23T11:17:37.370616Z [info] setup plugin alembic.autogenerate.schemas [alembic.runtime.plugins] loc=plugins.py:37
2026-06-23T11:17:37.377877Z [info] setup plugin alembic.autogenerate.tables [alembic.runtime.plugins] loc=plugins.py:37
2026-06-23T11:17:37.378617Z [info] setup plugin alembic.autogenerate.types [alembic.runtime.plugins] loc=plugins.py:37
2026-06-23T11:17:37.379460Z [info] setup plugin alembic.autogenerate.constraints [alembic.runtime.plugins] loc=plugins.py:37
2026-06-23T11:17:37.380464Z [info] setup plugin alembic.autogenerate.defaults [alembic.runtime.plugins] loc=plugins.py:37
2026-06-23T11:17:37.381155Z [info] setup plugin alembic.autogenerate.comments [alembic.runtime.plugins] loc=plugins.py:37
2026-06-23T11:17:37.382332Z [info] DAG bundles loaded: dags-folder [airflow.dag_processing.bundles.manager.DagBundlesManager] loc=manager.py:289
2026-06-23T11:17:38.197449Z [info] Filling up the DagBag from /opt/airflow/dags [airflow.dag_processing.dagbag.BundleDagBag] loc=dagbag.py:469
2026-06-23T11:17:38.323159Z [warning] The "airflow.providers.python.PythonOperator" attribute is deprecated. Please use "airflow.providers.standard.operators.python.PythonOperator". [py.warnings] category=UserWarning filename=/opt/airflow/dags/exercise_6_dag.py lineno=4
2026-06-23T11:17:38.373825Z [info] Bulk-writing dags to db [airflow.serialization.definitions.dag] count=1 loc=dag.py:197
2026-06-23T11:17:38.389918Z [info] Creating DAG for exercise_6_csv_processing [airflow.dag_processing.collection] loc=collection.py:182
2026-06-23T11:17:38.397149Z [info] get next_dagrun_info_v2 [airflow.serialization.definitions.dag] last_automated_run_info=None loc=dag.py:420 next_info=None
2026-06-23T11:17:38.397902Z [info] setting next_dagrun info [airflow.models.dag] loc=dag.py:792 next_dagrun=None next_dagrun_create_after=None next_dagrun_data_interval_end=None next_dagrun_data_interval_start=None next_dagrun_partition_date=None next_dagrun_partition_key=None
airflow@e380ea5772f7:/opt/airflow/dags$
```

In Airflow UI, execute the tasks by clicking trigger:

The screenshot shows the Airflow web interface. On the left, a sidebar contains navigation links: Home, Dags, Assets, Browse, and Admin. The 'Dags' section is active, showing a list of DAGs: 'create_csv', 'read_csv', 'calculate_revenue', and 'create_summary'. The 'calculate_revenue' DAG is selected, and its details are shown on the right. The 'Dag Run' section shows a manual trigger for 'manual_2026-06-23T11:18:17.268973+00:00' with a status of 'Success'. Below this, a table lists the task instances for this DAG run. The table has columns: Task ID, Map Index, State, Start Date, Try Number, Operator, Duration, and a set of icons for each task instance. The task instances are: 'create_summary' (Success, 2026-06-23 16:48:26, 1, PythonOperator, 00:00:00.145), 'calculate_revenue' (Success, 2026-06-23 16:48:25, 1, PythonOperator, 00:00:00.169), 'read_csv' (Success, 2026-06-23 16:48:23, 1, PythonOperator, 00:00:01.182), and 'create_csv' (Success, 2026-06-23 16:48:18, 1, PythonOperator, 00:00:08.453).

Task ID	Map Index	State	Start Date	Try Number	Operator	Duration
create_summary		Success	2026-06-23 16:48:26	1	PythonOperator	00:00:00.145
calculate_revenue		Success	2026-06-23 16:48:25	1	PythonOperator	00:00:00.169
read_csv		Success	2026-06-23 16:48:23	1	PythonOperator	00:00:01.182
create_csv		Success	2026-06-23 16:48:18	1	PythonOperator	00:00:08.453

Displaying output in Killercode:

```
airflow@e380ea5772f7:/opt/airflow/dags$ cat /tmp/sales_summary.txt
Laptop = 140000
Mouse = 2500
Keyboard = 3600
Total Revenue = 146100
airflow@e380ea5772f7:/opt/airflow/dags$
```